

SONA COLLEGE OF TECHNOLOGY

(An Autonomous Institution)

Department of Electrical and Computer Engineering

Mini Project Report

U23EF407 – DATABASE MANAGEMENT SYSTEMS

LABORATORY



Academic Year	2025-2026 (Even)
Year / Sem / Section	II / IV / A
Project Title	Electricity Billing System
Reg No	61782324113034, 61782324113035, 61782324113036, 61782324113037
Name of the Student	Logesh M, Lokesh RV, Mathivanan E, Mathivanan J
Faculty Name	Mr.G.ABDULKALAMAZAD, AP/CSE
Signature	

TABLE OF CONTENTS

S.No	Title	Page No
1	Abstract	03
2	Introduction	04
3	System Analysis 3.1 Hardware Requirements 3.2 Software Requirements	05
4	Modules 4.1 Title of the modules 4.2 Description of modules	06
5	System Design 5.1 System Architecture	07
6	System Implementation	08
7	Coding	10
8	Screen Shots	14
9	Conclusion	16

EX.NO:10

Electricity Billing System

DATE:06/04/26

1. ABSTRACT

The Electricity Billing System is developed to automate the process of calculating electricity bills and managing customer data efficiently. This system eliminates the need for manual calculations, thereby reducing errors and ensuring accurate billing based on actual electricity consumption. It uses Java as the front-end application language and MySQL as the back-end database management system. The system maintains complete records of customers, meter readings, generated bills, and payment transactions in an organised and reliable manner. It allows administrators to add new customers, enter meter readings, calculate units consumed, generate bills automatically based on applicable tariff rates, and update payment status upon settlement. The system improves operational efficiency, provides a clear audit trail, and supports report generation for monitoring billing and payment activities.

2. INTRODUCTION

Electricity billing is a fundamental operation in utility management. In traditional systems, billing officers manually record meter readings, compute consumption, apply tariff rates, and prepare bills by hand. This process is slow, prone to arithmetic errors, and difficult to audit or verify. As the number of consumers grows, manual billing becomes increasingly unmanageable.

This project develops a computerised Electricity Billing System using Java and MySQL to automate and streamline the entire billing lifecycle. The system allows an administrator to register customers, record monthly meter readings, auto-calculate units consumed and bill amounts based on tariff slabs, generate bill records, and track payment status. By replacing manual effort with an automated database-driven solution, the system ensures accuracy, speed, and reliability in electricity billing operations.

3. SYSTEM ANALYSIS

3.1 Hardware Requirements

- PC / Laptop
- Minimum 4 GB RAM
- Basic Input / Output Devices (Keyboard, Mouse, Monitor)

3.2 Software Requirements

- Java (JDK 17 or above) – Frontend and Application Logic
- MySQL 8.0 – Database Management
- JDBC Driver (mysql-connector-java) – Java-Database Connectivity
- IDE – Eclipse or NetBeans

4. MODULES

4.1 Title of the Modules

1. Dashboard
2. Customer
3. Billing
4. Bill History

4.2 Description of Modules

Dashboard: This module is used to add and manage total number of customers. The administrator can enter new customer details such as name, address, meter number and contact number.

Customer: This module is used to manage the customer list. And also add new customer like name, meter number, phone number and email.

Billing : The billing module is used to calculate and generate the electricity bill. This billing amount calculation is based on customer (meter number), billing month, units consumed.

Bill History: It is helpful to use view all generated bills and payment status.

5. SYSTEM DESIGN

5.1 System Architecture

The Electricity Billing System follows a two-tier client-server architecture. The presentation and business logic are handled by Java (Swing/AWT), while MySQL manages all persistent data storage. The two tiers communicate through JDBC (Java Database Connectivity).

- **Frontend:** Java (User Interface and Application Logic)
- **Backend:** MySQL Database

Process Flow:

Admin Login → Customer Registration → Meter Reading Entry → Unit Calculation → Bill Generation → Payment Recording → Database Update

6. SYSTEM IMPLEMENTATION

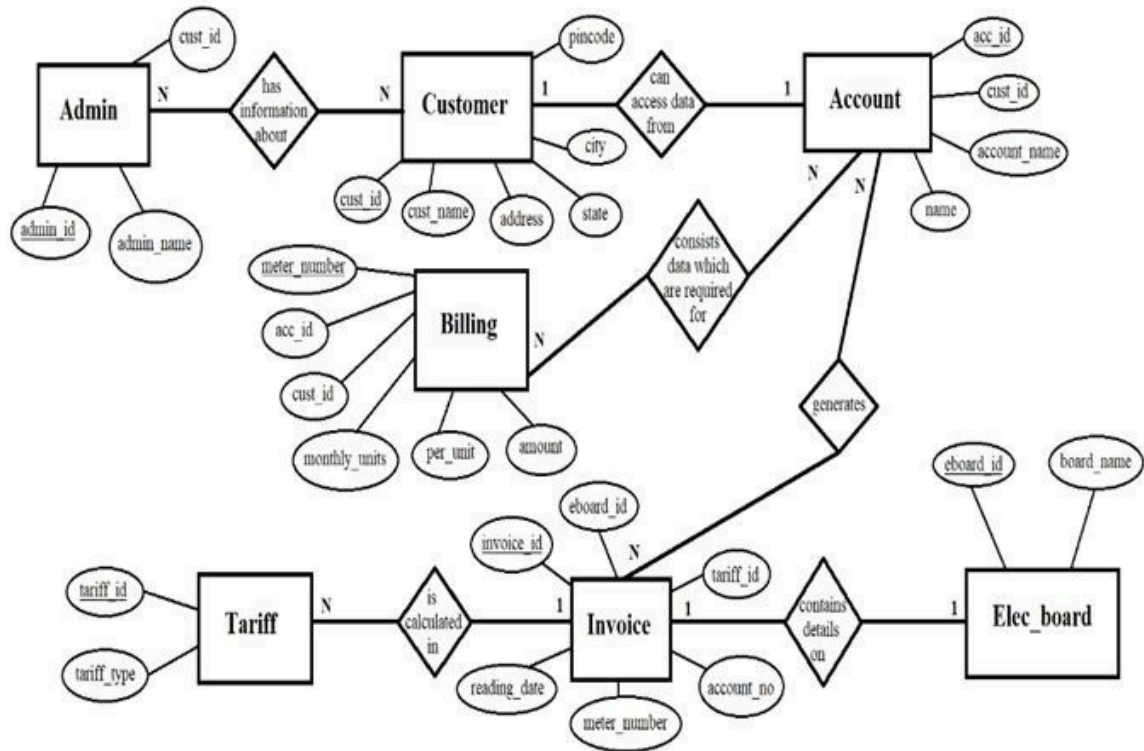
The system is implemented using the Java programming language connected to a MySQL database through JDBC. The application is built with Java Swing for the graphical user interface and uses PreparedStatement for secure database operations. The MySQL database, named electricity_db, consists of four tables: Customers, Meter, Billing, and Payment. The implementation performs the following steps:

1. Admin logs in and registers a new customer with name, address, and phone number.
2. Previous and current meter readings are entered for the selected customer.
3. Units consumed are calculated automatically:

$$\text{Units} = \text{Current Reading} - \text{Previous Reading}$$

4. Bill amount is computed based on the applicable tariff slab (e.g., ₹1.50/unit for 0–100 units, ₹5.00/unit for 100–300 units, and ₹8.00/unit above 300 units, ₹12.00/unit).
5. The generated bill is stored in the Billing table with status set to PENDING and a payment due date.
6. Upon payment, the admin records the transaction in the Payment table and the bill status is updated to PAID.

ER Diagram



7.CODING

Table creation

1. Create the Customers Table

```
CREATE TABLE IF NOT EXISTS customers (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    meter_number VARCHAR(255) UNIQUE NOT NULL,  
    name VARCHAR(255),  
    address VARCHAR(255),  
    phone VARCHAR(255),  
    email VARCHAR(255)  
);
```

2.Create the Bills Table

```
CREATE TABLE IF NOT EXISTS bills (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    meter_number VARCHAR(255),  
    customer_name VARCHAR(255),  
    billing_month VARCHAR(255),  
    units_consumed INT,  
    total_amount DOUBLE,  
    status VARCHAR(50) DEFAULT 'UNPAID',  
    generation_date DATE  
);
```

Customer

```
package com.billing.model;  
  
import jakarta.persistence.*;  
  
import lombok.Data;  
  
@Entity  
@Table(name = "customers")  
@Data
```



```
public class Customer {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
    @Column(unique = true, nullable = false)  
    private String meterNumber;  
    private String name;  
    private String address;  
    private String phone;  
    private String email;  
}
```

Billing

```
package com.billing.model;  
import jakarta.persistence.*;  
import lombok.Data;  
import java.time.LocalDate;  
@Entity  
@Table(name = "bills")  
@Data  
public class Bill {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
    private String meterNumber;  
    private String customerName;  
    private String billingMonth;  
    private Integer unitsConsumed;  
    private Double totalAmount;  
    private String status; // PAID, UNPAID  
    private LocalDate generationDate;  
}
```

Function for calculating bill

```
package com.billing.service;
import com.billing.model.Bill;
import com.billing.repository.BillRepository;
import com.billing.repository.CustomerRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.time.LocalDate;

@Service
public class BillingService {

    @Autowired
    private BillRepository billRepository;
    @Autowired
    private CustomerRepository customerRepository;

    public Bill generateBill(String meterNumber, String month, Integer units) {
        double amount = calculateAmount(units);
        Bill bill = new Bill();
        bill.setMeterNumber(meterNumber);
        customerRepository.findByMeterNumber(meterNumber).ifPresent(c -> {
            bill.setCustomerName(c.getName());
        });
        bill.setBillingMonth(month);
        bill.setUnitsConsumed(units);
        bill.setTotalAmount(amount);
        bill.setStatus("UNPAID");
        bill.setGenerationDate(LocalDate.now());
        return billRepository.save(bill);
    }

    private double calculateAmount(int units) {
        double amount = 0;
        if (units <= 100) {
            amount = units * 5;
        } else if (units <= 300) {
```

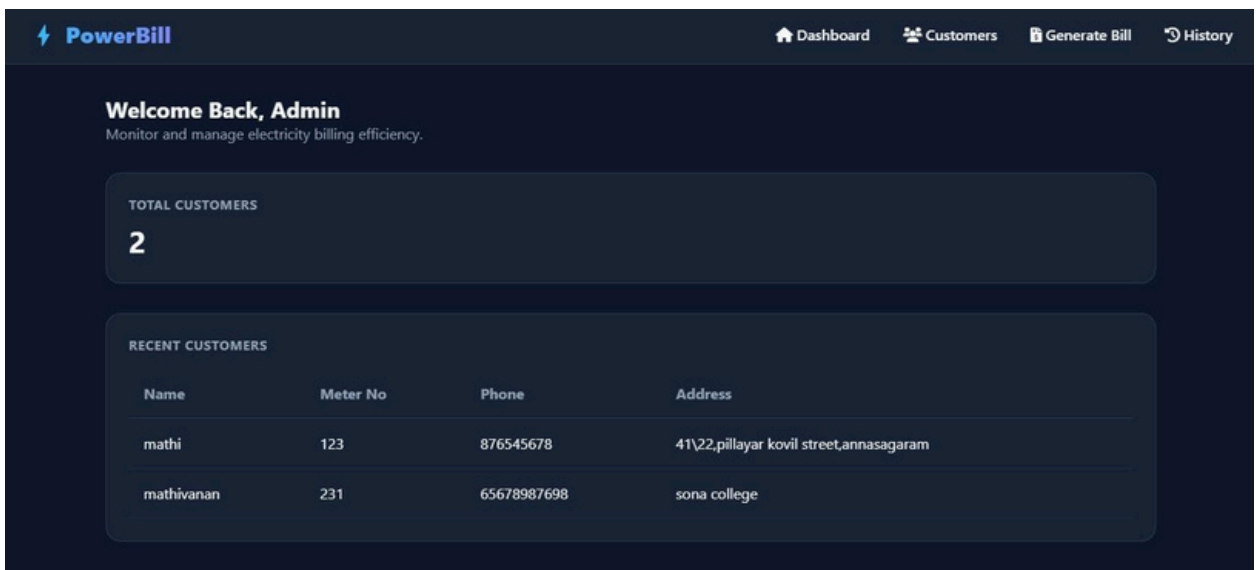
```

        amount = (100 * 5) + (units - 100) * 8;
    } else {
        amount = (100 * 5) + (200 * 8) + (units - 300) * 12;
    }
    return amount;
}
}

```

8. SCREEN SHOTS

Admin (Dashboard)



PowerBill Dashboard Customers Generate Bill History

Welcome Back, Admin
Monitor and manage electricity billing efficiency.

TOTAL CUSTOMERS
2

RECENT CUSTOMERS

Name	Meter No	Phone	Address
mathi	123	876545678	41\22,pillayar kovil street,annasagaram
mathivanan	231	65678987698	sona college

Customer registration Page

DashboardCustomersGenerate BillHistory

Customer Management

Add New Customer

Full Name

Meter Number

Phone

Email

CUSTOMER LIST

Name	Meter No	Phone
mathi	123	876545678
mathivanan	231	65678987698

Bill Generation Page

DashboardCustomersGenerate BillHistory

Generate Monthly Bill

Select Customer (Meter No)

-- Choose Meter --

Billing Month

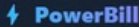
January

Units Consumed

0

Calculate and Generate

Billing History Page



[Dashboard](#)[Customers](#)[Generate Bill](#)[History](#)

Billing History

View all generated bills and their payment status.

Customer	Month	Units	Amount	Status	Date Generated
mathivanan	May	200	₹ 1300.0	UNPAID	2026-04-11

9. CONCLUSION

The Electricity Billing System successfully automates the complete billing process and significantly reduces manual effort in utility management. The system ensures accurate calculation of electricity units consumed and bill amounts by applying predefined tariff rules, eliminating arithmetic errors that are common in manual billing. By integrating Java for the application logic and MySQL for database storage, the system provides a reliable, structured, and easily maintainable solution.

The project demonstrates core DBMS concepts including relational table design, foreign key constraints, SQL operations such as INSERT, UPDATE, and SELECT, and JDBC connectivity between Java and MySQL. All five modules – Customer, Meter Reading, Billing, Payment, and Admin – were implemented and tested successfully. The system can be further enhanced in the future with features such as online payment integration, SMS or email notifications to customers, and graphical dashboards for consumption trend analysis.